# From a Go converter to a composed architecture document.

A library caller supplies HTML, chooses the document contract, and receives deterministic PDF bytes without shelling out to a browser or a system binary.

**Core idea.** The public API is a narrow ownership boundary around settings, page objects, conversion context, and output bytes. Internal layout and PDF stages remain replaceable behind that boundary.

server-generated HTML | context-aware | PDF bytes in memory

### 1 · Configure

Global page geometry, margins, background, and local-file policy.

### 2 · Add object

Attach one or more HTML pages with object-level overrides.

### 3 · Convert

Thread context through load, parse, style, layout, paint, and write.

### 4 · Consume

Read stable PDF bytes and store or stream them from the host service.

→ → →

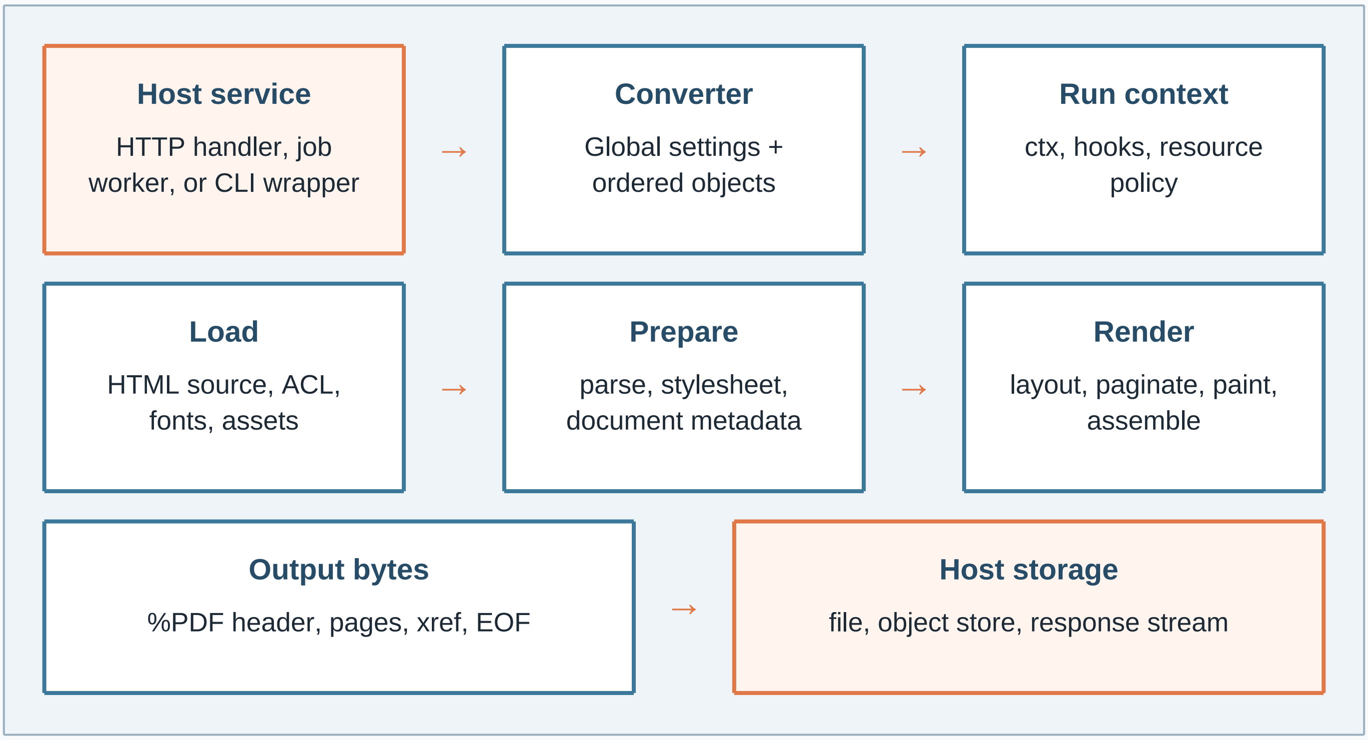
**5**
architecture pages in this example

**0**
external browser processes

**1**
explicit output ownership boundary

# The request path is explicit at every seam.

The converter owns configuration and output. Each page object owns its source and local overrides. The context gives callers a cancellation and deadline boundary.




### Ownership rules

- `AddObject` copies public settings before conversion.
- `Output` returns a copy of the completed bytes.
- A converter is used by one conversion flow at a time.
- Failures return as errors before partial output is published.

### Signals exposed to callers

- Context cancellation and deadlines.
- Phase names and integer progress callbacks.
- Info, warning, and error conversion hooks.
- Typed sentinel errors for invalid boundaries.

**Design note.** A library caller never needs to know which internal package performs a stage. The public contract stays useful even as layout and PDF implementation details evolve.

# Rendering is a staged document pipeline.

The renderer turns one source document into an ordered display model. Each stage contributes a stronger invariant and keeps its work visible to the next stage.

| Stage | Input → output | Invariant carried forward |
|---|---|---|
| **Load** | URL/path/inline bytes → source tree | ACL checks and resource policy are applied before fetches. |
| **Parse** | HTML/CSS text → nodes and rules | Malformed but recoverable markup remains traversable. |
| **Style** | rules + ancestry → resolved styles | Inherited values, custom properties, and media are resolved once. |
| **Layout** | styled tree → boxes and flow positions | Widths, heights, text runs, tables, flex, and grid have geometry. |
| **Paint** | boxes → display operations | Operations preserve order, links, backgrounds, and text metadata. |
| **Write** | operations → PDF bytes | Pages, resources, xref, and EOF form a readable PDF artifact. |

## Display-list handoff

BOX TREE
**Document geometry**
x, y, width, height, page ownership

→

DISPLAY LIST
**Paint operations**
fill, stroke, image, text, link, transform

→

PDF WRITER
**Serialized pages**
fonts, resources, content streams

### Why a display list?

Pagination can move a group of operations without rebuilding source nodes. Tests can inspect paint order and page ownership before serialization.

### Why keep the API above it?

Callers need a document result, not a renderer vocabulary. The public surface can remain small while internal contracts stay testable.

# Safe defaults make the library embeddable.

A service embedding the converter must be able to decide which inputs can read local files, which resources can be fetched, and when a request should stop.

ACL

### Local resources

Local-file access is blocked by default. The global enable flag and object-level unblock form an explicit pair.

CTX

### Cancellation

The caller supplies a context. Load, conversion, and output stages can stop without leaving a published partial PDF.

LIMITS

### Resource bounds

Object, copy, page, stylesheet, and source limits prevent accidental amplification from untrusted inputs.

```
converter:= gowkhtmltopdf.NewConverter()
converter.Global().Set("enablelocalfileaccess", "true")
page:= gowkhtmltopdf.NewObjectSettings().SetPage(inputPath)
page.Set("load.blocklocalfileaccess", "false")
converter.AddObject(page)
err:= converter.Convert(ctx)
```

## Failure boundaries

| Condition | Observed by host | Recommended host action |
|---|---|---|
| Missing page object | typed no-page error | return a 4xx configuration response |
| Blocked local file | load/resource error | keep ACL blocked and report the source |
| Context cancellation | context error | stop work and do not publish output |
| Invalid setting | setting validation error | reject the request before conversion |

internal stage   caller decision   published contract

# One small example can carry a complete document contract.

The generator beside this template is intentionally ordinary Go: configure, add a local page, convert with a context, validate the page count, and write the bytes.

**HTML template**
architecture-diagram.html

+

**Go program**
generate.go / public API

=

**Golden artifact**
architecture-diagram.pdf

### Integration checklist

1. Choose whether input is a path or in-memory body.
2. Set page geometry and rendering policy explicitly.
3. Pair local-file settings when local resources are intended.
4. Pass the host request context to `Convert`.
5. Write or stream `Output` only after success.

### What this artifact demonstrates

- Five stable, explicit document pages.
- Architecture diagrams built from print-safe CSS boxes.
- Text, tables, code, metadata, and page breaks.
- A reproducible output path beside its source template.

**Closing contract.** The HTML remains understandable as a document, the Go program remains understandable as an integration example, and the PDF is a reviewable artifact produced by the same public API that an embedding service would call.

HTML in · Go API · PDF out